

计算机图形学期末复习资料

计算机图形学期末复习资料

一、模型视图变换矩阵（Model-View Transformation）

1.1 齐次坐标（Homogeneous Coordinates）

- 点 (Point): 四维坐标 $(x, y, z, 1)$ ，第四维为1
- 向量 (Vector/Direction): 四维坐标 $(x, y, z, 0)$ ，第四维为0
- 为什么用齐次坐标? 统一表示平移、旋转、缩放等所有仿射变换，矩阵乘法可复合

1.2 基本变换矩阵（必须手撕！）

平移矩阵 $T(dx, dy, dz)$

$$T = \begin{vmatrix} 1 & 0 & 0 & dx \\ 0 & 1 & 0 & dy \\ 0 & 0 & 1 & dz \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

- 三个自由度，沿各轴移动

缩放矩阵 $S(sx, sy, sz)$

$$S = \begin{vmatrix} sx & 0 & 0 & 0 \\ 0 & sy & 0 & 0 \\ 0 & 0 & sz & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

- 原点为不动点，沿各轴伸缩

绕Z轴旋转矩阵 $R_z(\theta)$

$$R_z = \begin{vmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

- 从Z轴正方向看，逆时针为正

绕X轴旋转矩阵 $R_x(\theta)$

$$R_x = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

绕Y轴旋转矩阵 $R_y(\theta)$

$$R_y = \begin{pmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

注意 R_y 的符号！ 与 R_x 、 R_z 不同，因为要保持右手坐标系

1.3 变换复合 (Transformation Concatenation)

- 顺序不可交换！ 矩阵乘法不满足交换律
- 从右到左应用： $p' = ABCp$ 表示先应用A，再B，再C（A在最右）
- 绕任意点旋转： $M = T(pf) \cdot R(\theta) \cdot T(-pf)$
 1. 平移不动点到原点
 2. 旋转
 3. 平移回去

1.4 欧拉角 (Euler Angles)

- 绕任意轴旋转 = $R_z(\alpha) \cdot R_y(\beta) \cdot R_x(\gamma)$
- 顺序很重要！ 调换顺序需调整角度

1.5 OpenGL相关函数

```
glMatrixMode(GL_MODELVIEW); // 设置当前矩阵为模型视图矩阵
glLoadIdentity();           // 重置为单位矩阵
glTranslatef(dx, dy, dz);    // 平移
glRotatef(angle, x, y, z);   // 绕轴(x,y,z)旋转angle度
glScalef(sx, sy, sz);        // 缩放
gluLookAt(eyeX, eyeY, eyeZ, atX, atY, atZ, upX, upY, upZ); // 相机设置
```

二、视图与投影变换 (Viewing & Projection)

2.1 相机设置 gluLookAt

```
void gluLookAt(eyex, eyey, eyez,    // 视点位置
               atx, aty, atz,       // 被观察点
               upx, upy, upz);      // 向上向量
```

- 几何意义：确定相机的位置和朝向
- Up向量不平行于投影平面时：需要重新计算正交基，通过叉积构建相机坐标系

2.2 正交投影 glOrtho

```
void glOrtho(left, right, bottom, top, near, far);
```

- 视景体：与坐标轴平行的长方体
- 规范化矩阵推导（重点！）：
将 $[left, right] \times [bottom, top] \times [near, far]$ 映射到 $[-1, 1]^3$

步骤：

1. 平移中心到原点： $T(-(left+right)/2, -(bottom+top)/2, -(near+far)/2)$
2. 缩放： $S(2/(right-left), 2/(top-bottom), 2/(far-near))$

组合矩阵：

$$M = \begin{vmatrix} 2/(r-l) & 0 & 0 & -(r+l)/(r-l) \\ 0 & 2/(t-b) & 0 & -(t+b)/(t-b) \\ 0 & 0 & 2/(f-n) & -(f+n)/(f-n) \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

2.3 透视投影

glFrustum

```
void glFrustum(left, right, bottom, top, near, far);
```

- 通用透视投影，可定义非对称视景体
- $near, far > 0$ （必须为正！与正交投影不同）

gluPerspective（更常用）

```
void gluPerspective(fovy, aspect, near, far);
```

- fovy：垂直视角（度）
- aspect：宽高比
- 只能定义对称视景体

透视变换转成正交变换：先透视除法，再正交投影

2.4 视口变换 glViewport

```
void glViewport(x, y, width, height);
```

- 将规范化坐标[-1,1]映射到窗口坐标
- 公式：
 - $WinX = 0x + (x+1)/2 * width$
 - $WinY = 0y + (y+1)/2 * height$
 - $WinZ = (z+1)/2$ （深度缓冲用，范围[0,1]）

2.5 顶点变换完整流程

对象坐标 → 模型视图变换 → 相机坐标 → 投影变换 → 裁剪坐标 → 透视除法 → 规范化设备坐标 → 视口变换 → 窗口坐标

要求能手算一个顶点经过完整变换后的屏幕坐标！

三、光照计算（Phong/Blinn光照模型）

3.1 光源类型

类型	特点	OpenGL设置
点光源	有位置，向各方向均匀发光	w=1.0
平行光（无穷远）	方向恒定，无位置	w=0.0
聚光灯	锥形照明范围，有方向、角度、衰减	SPOT_CUTOFF等
环境光	均匀照亮场景，无方向	全局设置

3.2 材料表面类型

- 镜面（Specular）：光滑表面，高光集中
- 漫反射（Diffuse）：粗糙表面，光向各方向散射

- 透明 (Translucent)：光可穿透，有折射

3.3 Phong光照模型（必须会算！）

四个关键向量

- **n**：表面法向量（单位化）
- **l**：入射光方向（指向光源，单位化）
- **v**：视点方向（指向观察者，单位化）
- **r**：理想反射方向， $r = 2(l \cdot n)n - l$

三个分量

1. 环境光 (Ambient)

$$I_a = k_a \cdot L_a$$

- k_a ：环境光反射系数
- L_a ：环境光强度

2. 漫反射 (Diffuse) - Lambertian

$$I_d = k_d \cdot (l \cdot n) \cdot L_d$$

- 实际使用 $\max(l \cdot n, 0)$ ，负值取0
- 与视点位置无关！

3. 镜面反射 (Specular)

$$I_s = k_s \cdot (v \cdot r)^\alpha \cdot L_s$$

- α ：高光系数 (shininess)
 - 100-200：金属
 - 5-10：塑料
 - ∞ ：镜子

综合公式（单个光源）

$$I = I_a + I_d + I_s = k_a \cdot L_a + k_d \cdot \max(l \cdot n, 0) \cdot L_d + k_s \cdot (v \cdot r)^\alpha \cdot L_s$$

多光源叠加

对每个光源计算后求和，RGB分别计算

3.4 Blinn-Phong改进模型（OpenGL实现）

- **问题：** Phong需要计算反射向量 r ，计算量大
- **改进：** 用半角向量 h 代替 r

```
h = (l + v) / |l + v| // 入射光和视点的角平分线
```

- 镜面项改为： $I_s = k_s \cdot (n \cdot h)^\beta \cdot L_s$
- β 适当选取以匹配Phong的 α
- 当 l, n, v 共面时： $\psi = \phi/2$ ，即半角是反射角的一半

3.5 距离衰减

```
衰减因子 = 1 / (a + b·d + c·d²)
```

- a ：常数项， b ：线性项， c ：二次项
- 默认： $a=1, b=0, c=0$

3.6 OpenGL光照设置

```
// 启用光照
glEnable(GL_LIGHTING);
glEnable(GL_LIGHT0);

// 设置光源属性
glLightfv(GL_LIGHT0, GL_POSITION, pos); // 位置/方向
glLightfv(GL_LIGHT0, GL_AMBIENT, ambient); // 环境光
glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse); // 漫反射
glLightfv(GL_LIGHT0, GL_SPECULAR, specular); // 镜面光

// 设置材料属性
glMaterialfv(GL_FRONT, GL_AMBIENT, mat_ambient);
glMaterialfv(GL_FRONT, GL_DIFFUSE, mat_diffuse);
glMaterialfv(GL_FRONT, GL_SPECULAR, mat_specular);
glMaterialf(GL_FRONT, GL_SHININESS, shine); // 高光系数
```

四、多边形明暗处理（Shading）

4.1 Flat Shading（平面着色）

- 整个多边形用一个颜色
- 取第一个顶点（或某个指定顶点）的颜色
- 快，但多边形边界明显
- OpenGL: `glShadeModel(GL_FLAT)`

4.2 Gouraud Shading（高洛德着色）

- 对顶点颜色进行双线性插值
- 步骤：
 1. 计算每个顶点的法向量（相邻面法向平均）
 2. 用光照模型计算每个顶点颜色
 3. 对多边形内部像素颜色插值
- 高光可能丢失（如果高光不在顶点）
- OpenGL 默认: `glShadeModel(GL_SMOOTH)`

4.3 Phong Shading（冯氏着色）

- 对法向量进行双线性插值，再计算光照
- 步骤：
 1. 计算每个顶点法向量
 2. 对多边形内部像素法向量插值
 3. 对每个像素用法向量计算光照
- 效果更好，更光滑，但计算量大（6-8倍）
- 可用片段处理器实现

4.4 法向量计算

```
// 三角形法向量（叉积）
n = (p1 - p0) × (p2 - p0)
n = n / |n| // 归一化

// 顶点法向量（平均）
N_vertex = (N_face1 + N_face2 + ... + N_facek) / k
N_vertex = N_vertex / |N_vertex|
```

五、Bezier曲线（手撕重点！）

5.1 基本概念

- **n次Bezier曲线**：需要 $n+1$ 个控制点
- **性质**：
 - 经过起点 P_0 和终点 P_n
 - 起点切线方向： $P_0 \rightarrow P_1$ ， 终点切线方向： $P_{n-1} \rightarrow P_n$
 - 凸包性：曲线在控制点凸包内
 - 仿射变换不变性

5.2 de Casteljau算法（递归手撕）

线性Bezier（1次，2个控制点）

$$B(t) = (1-t) \cdot P_0 + t \cdot P_1, \quad t \in [0, 1]$$

二次Bezier（2次，3个控制点）

$$B(t) = (1-t)^2 \cdot P_0 + 2(1-t)t \cdot P_1 + t^2 \cdot P_2$$

几何构造：

1. $D = (1-t)A + tB$ ($A=P_0, B=P_1$)
2. $E = (1-t)B + tC$ ($C=P_2$)
3. $F = (1-t)D + tE \leftarrow$ 曲线上的点

三次Bezier（3次，4个控制点）

$$B(t) = (1-t)^3 \cdot P_0 + 3(1-t)^2 t \cdot P_1 + 3(1-t)t^2 \cdot P_2 + t^3 \cdot P_3$$

组合系数规律：二项式系数！

- 3次：(1,3,3,1)
- 系数之和 = 1

几何构造：

1. 对4个控制点两两线性插值 $\rightarrow$ 3个点
2. 对3个点两两线性插值 $\rightarrow$ 2个点
3. 对2个点线性插值 $\rightarrow$ 1个点（曲线上的点）

5.3 n次Bezier一般公式

$$B(t) = \sum_{i=0}^n C(n,i) \cdot (1-t)^{n-i} \cdot t^i \cdot P_i$$

其中 $C(n,i)$ 是组合数, Bernstein基函数

5.4 Bezier曲线拼接

- 控制点过多 → 自由度差, 难控制
- 解决方案: 分成多段三次Bezier拼接
- 连续性:
 - C^0 连续: 前段终点 = 后段起点 (位置连续)
 - C^1 连续: 切线方向相同, 大小成比例 (一阶导连续)
 - C^2 连续: 曲率连续 (二阶导连续)

5.5 Bezier曲面

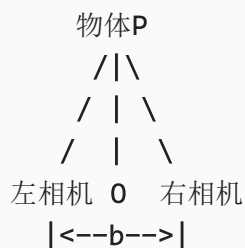
- 给定 $(n+1) \times (m+1)$ 控制点网格
- 先生成 n 次Bezier曲线 (行方向)
- 再对曲线上的点生成 m 次Bezier曲线 (列方向)
- 公式: $B(u,v) = \sum \sum B_{i,n}(u) \cdot B_{j,m}(v) \cdot P_{ij}$

六、视差求距离 (双目视觉)

6.1 基本原理

- 两个相机 (左眼、右眼) 拍摄同一场景
- 同一物点在两个图像上的位置不同 → 视差 (Disparity)

6.2 几何模型



焦距 f , 基线距离 b

6.3 距离公式推导

设：

- 左相机坐标系原点
- 右相机在x轴上，距离b
- 两相机光轴平行（简化模型）
- 焦距f

物点P在左图像坐标： $x_L = f \cdot X / Z$

物点P在右图像坐标： $x_R = f \cdot (X - b) / Z$

视差： $d = x_L - x_R = f \cdot b / Z$

距离： $Z = f \cdot b / d$

6.4 关键结论

- 视差与距离成反比：近处物体视差大，远处物体视差小
- 3D电影原理：利用偏振光或色差让左右眼看到不同图像，大脑合成立体感

七、Constructive Solid Geometry（CSG）

7.1 基本概念

- CSG：用简单体素通过集合运算构造复杂物体
- 体素（Primitives）：球、立方体、圆柱、圆锥等

7.2 集合运算（重点！）

运算	符号	含义	结果
并集	U 或 +	$A \cup B$	在A或B中的点
交集	$\cap$ 或 $\times$	$A \cap B$	同时在A和B中的点
差集	- 或 \	$A - B$	在A但不在B中的点
补集	' 或 $\neg$	A'	不在A中的点

7.3 隐式表示

每个体素用隐式函数表示：

- 球: $f(x,y,z) = x^2 + y^2 + z^2 - r^2 \leq 0$ (内部)
- 立方体: 各半空间的交集

7.4 运算实现

- 并集: $f_{A \cup B} = \min(f_A, f_B)$
- 交集: $f_{A \cap B} = \max(f_A, f_B)$
- 差集: $f_{A-B} = \max(f_A, -f_B)$

7.5 应用场景

- CAD/CAM
- 3D打印切片
- 物理仿真 (碰撞检测)

八、图形绘制流水线

8.1 光栅化流水线 (OpenGL)

顶点数据 → 顶点处理 (Vertex Shader) → 图元装配 → 裁剪 → 光栅化 → 片段处理 (Fragment Shader) → 帧缓冲

各阶段任务

阶段	任务
顶点处理	坐标变换 (模型视图、投影)、光照计算
图元装配	顶点组装成点、线、三角形
裁剪	去除视景体外部分
光栅化	生成片段 (潜在像素), 属性插值
片段处理	纹理映射、深度测试、颜色混合

8.2 隐藏面消除

- Z-Buffer (深度缓冲)

```
glEnable(GL_DEPTH_TEST); // 启用
glClear(GL_DEPTH_BUFFER_BIT); // 清空
```

- 保存每个像素的深度值
- 新片段更近则更新颜色和深度
- 背面剔除

```
glEnable(GL_CULL_FACE);  
glCullFace(GL_BACK); // 剔除背面
```

8.3 光线跟踪 (Ray Tracing)

- 与光栅化区别：逐像素而非逐物体
- Whitted-Style光线跟踪：
 1. 从相机发射Primary Ray
 2. 与场景求交（最近交点）
 3. 计算局部光照（Phong）
 4. 若表面镜面/透明，递归发射反射/折射光线
 5. 发射Shadow Ray检测阴影
- 加速结构：均匀网格、Octree、KD-Tree、BVH

九、OpenGL编程要点

9.1 程序结构

```
#include <GL/glut.h>  
  
void init() {  
    glClearColor(0.0, 0.0, 0.0, 1.0); // 背景色  
    glColor3f(1.0, 1.0, 1.0);          // 前景色  
    // 设置投影、视图...  
}  
  
void display() {  
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);  
    glBegin(GL_TRIANGLES);  
        glVertex3f(...);  
        glColor3f(...);  
    glEnd();  
    glutSwapBuffers(); // 双缓冲  
}  
  
int main(int argc, char** argv) {  
    glutInit(&argc, argv);
```

```
glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
glutInitWindowSize(800, 600);
glutCreateWindow("Title");
init();
glutDisplayFunc(display);
glutMainLoop(); // 事件循环
}
```

9.2 回调函数

函数	用途
glutDisplayFunc	窗口重绘
glutReshapeFunc	窗口大小改变
glutKeyboardFunc	键盘输入
glutMouseFunc	鼠标点击
glutMotionFunc	鼠标拖动（按下）
glutIdleFunc	空闲时调用（动画）

9.3 图元类型

- GL_POINTS , GL_LINES , GL_LINE_STRIP , GL_LINE_LOOP
- GL_TRIANGLES , GL_TRIANGLE_STRIP , GL_TRIANGLE_FAN
- GL_QUADS , GL_QUAD_STRIP , GL_POLYGON

十、几何表示方式

10.1 隐式 vs 显式

类型	定义	例子
隐式	满足关系 $f(x,y,z)=0$	球: $x^2+y^2+z^2=1$
显式	直接给出坐标	点云、网格
参数式	$x=x(u,v)$, $y=y(u,v)$, $z=z(u,v)$	Bezier曲面

10.2 其他表示

- 点云 (Point Cloud): 离散点集合

- **符号距离场 (SDF)**: 每点存储到表面的有向距离
- **网格 (Mesh)**: 顶点+面 (三角形/四边形)
- **细分曲面**: 从粗网格递归细分得到光滑曲面

十一、其他重点

11.1 颜色模型

- **RGB (加色系统)**: 显示器、投影仪, 红绿蓝叠加
- **CMY (减色系统)**: 打印、照片, 青品黄过滤
- **三原色理论**: 人眼三种视锥细胞 (S,M,L)

11.2 针孔相机模型

投影公式:
 $x_p = -d \cdot x / z$
 $y_p = -d \cdot y / z$

- d: 焦距 (针孔到胶片距离)
- 无穷景深, 视角 $\theta = 2 \cdot \arctan(h/2d)$

11.3 全局 vs 局部光照

	全局光照	局部光照
方法	光线跟踪、辐射度、路径跟踪	Phong/Blinn模型
效果	阴影、反射、折射、色渗	简单明暗
速度	慢 (离线渲染)	快 (实时)
硬件	一般CPU计算	GPU加速

11.4 动画基础

- **关键帧插值**: 在时间轴上插值位置/旋转/缩放
- **Blend Shape**: 通过混合多个形状变形
- **LBS (线性混合蒙皮)**: 骨骼动画, 顶点受多骨骼影响
- **质点-弹簧系统**: 模拟布料、柔体

附录：常用公式速查

向量运算

- 点积： $a \cdot b = |a| |b| \cos \theta = a_x \cdot b_x + a_y \cdot b_y + a_z \cdot b_z$
- 叉积： $a \times b = (a_y \cdot b_z - a_z \cdot b_y, a_z \cdot b_x - a_x \cdot b_z, a_x \cdot b_y - a_y \cdot b_x)$
- 归一化： $\hat{a} = a / |a|$

矩阵乘法

- $(AB)_{ij} = \sum_k A_{ik} \cdot B_{kj}$
- 注意：矩阵乘法不满足交换律！

反射向量

- $r = 2(l \cdot n)n - l$

半角向量

- $h = (l + v) / |l + v|$

视差距离

- $z = f \cdot b / d$

考试技巧

- 矩阵题**：先写齐次坐标，注意平移矩阵最后一列，确认矩阵维度
- 光照计算**：先画向量图，确认各向量方向，注意 $\max(0, \dots)$ 处理
- Bezier曲线**：记住de Casteljau递归，组合系数是二项式系数
- 视差题**：画双目几何图，确认相似三角形关系
- CSG**：明确体素隐式函数， $\min/\max$ 实现并交差
- OpenGL代码**：记住程序框架，init-display-main三段式

祝考试顺利！